

Winning the Pokémon TCG AI Battle Challenge: A 6.5-Week Technical Battle Plan (Both Categories)

TL;DR

- The simulator is the **full physical Pokémon TCG (Standard 2026 "H-On" / Mega Evolution era)**, **60-card decks**, **6 prizes** — NOT TCG Pocket — running on the University of Tokyo Matsuo Institute's "cabt Engine" for kaggle-environments; you submit a `submission.tar.gz` (`main.py` + `deck.csv` + engine `cg/`, ≤ 197.7 MiB) whose `agent(obs_dict) -> list[int]` returns option indices, ranked by a TrueSkill-like Gaussian ladder starting at $\mu = 600$. The Simulation ladder locks at the Aug 17 08:59 JST deadline (11:59 p.m. UTC Aug 16); the Strategy report is due Sep 14 08:59 JST (Sep 13 UTC).
- Recommended novel approach — **"BeliefSet-DMC"**: a Deep-Monte-Carlo policy (DouZero-style) with a **card-text-grounded set-transformer encoder** and an **opponent-belief head**, bootstrapped by behavior-cloning the daily Kaggle episode replays, refined by league/PFSP self-play, invoking **belief-weighted re-determinized ISMCTS with a Gumbel root** only at low-margin decisions via the engine's `search_begin`/`search_step` forward model, then distilled to a fast (~ 1 – 3 M-param) net for latency. The deck is co-optimized with the policy via MAP-Elites + double-oracle.
- Because the ladder locks ~ 4 weeks before the report, **front-load the agent to peak by Aug 16**, then spend the remaining time on the ablations, hypothesis tests, matchup matrices, and figures that actually win the 70/20/10 Strategy rubric. Ladder rank helps but does not guarantee a Strategy prize; a mid-tier agent with deep, well-visualized analysis can still finish top-8.

Key Findings

The competition (confirmed from the Kaggle pages, the official pokemon.co.jp site, and the cabt engine docs)

- **Two linked competitions.** The Simulation Category (slug `pokemon-tcg-ai-battle`) is a live auto-battle ladder with **no prize money on its own**; entering it is a prerequisite for the Strategy Category (slug `pokemon-tcg-ai-battle-challenge-strategy`), which holds the **\$240,000 prize pool** (confirmed by EdTech Innovation Hub quoting Kaggle: "the separate strategy category has a \$240,000 prize pool"). The top 8 Strategy teams each receive \$30,000 and advance to a live Tokyo final; per Dexerto, "the overall winner will earn \$50,000," the runner-up \$30,000, and finalists receive "\$3,000 in Google Cloud credits per participant."
- **Organizers.** Per Misprint, on "June 16, 2026, The Pokemon Company announced the Pokemon TCG AI Battle Challenge, built with the Japanese AI firm HEROZ and the Matsuo Institute, with support from Google, Google Cloud, NVIDIA, and Kaggle." AICU notes the cabt Engine was "developed by the University of Tokyo's Matsuo Institute specifically for the Kaggle environment."

- **Deadlines.** Per EdTech Innovation Hub, "Final simulation submissions are due by 11:59 p.m. UTC on August 16. Games will continue from August 17 until approximately August 31 ... before the final rankings are confirmed" (= Aug 17 08:59 JST cutoff). The Strategy entry/merger deadline is ~Sep 6 and the final report is due Sep 13 UTC (Sep 14 08:59 JST), judged through ~Oct 11. The Simulation entry/merger deadline is ~Aug 9.
- **Rubric (Strategy).** Model Score 70% — (a) clarity of the chosen approach and rationale, (b) originality and technical soundness, (c) consistency under repeated matches, (d) avoidance of over-reliance on specific initial states/matchups/situational advantages, (e) in-track performance; Deck Score 20% — deck-concept clarity + how effectively key cards are selected and utilized for the game plan; Report Score 10% — logical structure/writing + effectiveness of figures/charts/tables. The report is capped at 2,000 words with an attached media gallery. The organizers explicitly state a high ladder rank helps but does not guarantee a strong Strategy result and that mid/lower-tier teams can score highly through original analysis and structured reporting.

The engine and submission contract (confirmed from matsuoinstitute.github.io/cabt)

- **Format.** Full physical Pokémon TCG, "based on official rules but uniquely adjusted for the tournament"; the materials warn "some simulator behavior differs from the official Pokémon TCG rules." Standard-format card pool of ~2,000 cards. Decks are exactly 60 card IDs (`battle_start(deck0, deck1)` raises if not 60).
- **Agent contract.** `agent(obs_dict) -> list[int]` returns a list of indices into `obs_dict["select"]["option"]`, of length between `minCount` and `maxCount` (multi-select; `minCount` may be 0). On the initial deck-selection phase `current` and `select` are `None` and the agent returns its 60 card IDs. `battle_select(select_list: list[int])` is documented as taking a "List of selected option indices." The agent must never crash (always return a legal fallback) and must respect the time budget.
- **Observation** = `{logs, current, select}`. `current` (State) holds `players` (each `PlayerState`: `active`, `bench` ≤ 5 , `hand` [own cards visible; opponent shows count only], `prize` [face-down = `None`], `deckCount`, `discard`, `handCount`, and `poisoned/burned/asleep/paralyzed/confused` flags), `stadium`, and turn / first-player / supporter-stadium-energy-usage / retreat / result fields. `select` (SelectData) exposes `option: list[Option]`, `minCount`, `maxCount`, `type` (11 SelectTypes: MAIN, CARD, ENERGY, YES_NO, EVOLVE, ATTACK, SKILL, COUNT, SPECIAL_CONDITION, ...), `context` (49 SelectContexts), and `remainEnergyCost` / `remainDamageCounter` / `deck` / `contextCard` / `effect`. Options carry 17 OptionTypes (PLAY, ATTACH, EVOLVE, ATTACK, RETREAT, ABILITY, DISCARD, END, ENERGY, ...) with type-dependent target fields (area, index, cardId, attackId, energyIndex, ...).
- **Forward model for search.** `search_begin(agent_observation, your_deck, your_prize, opponent_deck, opponent_prize, opponent_hand, opponent_active, manual_coin=False)` returns a root `SearchState`; critically, **you must supply predicted/determinized card IDs for every hidden zone** (your deck order, both prize sets, opponent deck/hand/active) — this is precisely a determinization interface. `search_step(search_id, select)` advances one decision and returns the next `SearchState` (whose `.observation.select` yields the

next legal options to branch on), so iterating it is a full MCTS forward simulator.

`search_end()` and `search_release(search_id)` free/reuse memory; `manual_coin=True`

lets you fix coin outcomes for chance-node expansion.

- **Ladder mechanics.** 5 submissions/day; only the 2 most recent are scored; each submission first plays a validation game vs a copy of itself; agents enter at $\mu = 600$; a Gaussian/TrueSkill-like rating updates on win/loss/draw with **margin-independent** updates (blowouts and squeakers count the same). Time budget: AICU confirms "each player's time limit is set to a maximum of 10 minutes. Since running out of time results in an immediate loss," and Shane the Gamer confirms "any player who runs out the clock losing that game" — i.e., a **cumulative 600-second per-player clock**. A per-move limit exists but no official number is published. Per EdTech Innovation Hub, "Simulation entries must be submitted as a .tar.gz bundle no larger than 197.7 MiB, with main.py in the top-level directory and a deck.csv file included."

Empirical reality from the live ladder (community sources — directional, not official)

- On the official Kaggle discussion thread, a participant's timing analysis observes that most of the field is rule-based bots or copies of the public example agents; the very top player "loads a heavy model but also fully utilises the time" (likely RL + bounded search), while most other top agents appear to use no search. One RL competitor confirms an ~8-second load time for a 1.7M-param model. Several report that search "doesn't help" because the value head cannot reliably rank actions under high uncertainty — a textbook symptom of imperfect-information value ambiguity.
- Community repos report observed ladder μ from ~530 to ~880: the strongest known result is an official Dragapult pilot at ~880 μ ; a home-grown SearchScorer $\times$ Lucario reached ~660; an "RL+MCTS v5" regressed to ~580. A separate repo's best result was a **simple** Iono's Bellibolt ex Lightning deck at Elo 836, beating "stronger" combo decks; the author's explicit lesson: a rule-based/scoring agent pilots a simple deck cleanly and a complex one clunkily, so **keep decks simple and A/B-test on the real ladder** (local sims — both a ctypes harness and the official cabt env — mispredicted ladder rank).

The 2026 Standard "Mega era" meta (for deck choice)

- Per Card Chill's rotation analysis, The Pokémon Company announced the 2026 Standard rotation on January 9, 2026, "banishing all G regulation mark cards" — effective on Pokémon TCG Live March 26 with in-person enforcement April 10 — which "wiped ~20% of the card pool," removing 2025 pillars including Gholdengo ex (15.68% pre-rotation meta share), Iono, Arven, Professor's Research, Nest Ball, Counter Catcher, Pidgoot ex, Gardevoir ex, and Charizard ex (Obsidian Flames). The Mega Evolution series introduced 300–340+ HP Pokémon ex that evolve directly from a basic ex.
- Post-rotation top archetypes: **Dragapult ex / Dusknoir** (Phantom Dive: 200 + 5 bench counters, with Dusknoir's Sinister Hand moving counters for precise KOs) — per PokeBeach's Zakary Krekeler (Mar 28, 2026), it "has dominated the initial post-rotation format to the tune of meta shares around 25%," and Grant Manley (Apr 2, 2026) notes "Japanese tier lists have had it by itself in the top tier, tiers above every single other deck." Behind it: **Mega Lucario ex** (340 HP, Aura Jab + energy accel), **Mega Absol ex** (flexible

Dark toolbox), **Mega Gardevoir ex**, **Mega Charizard X ex**, **Mega Starmie ex** (bench-snipe spread), plus **Gholdengo** and single-prize/control lines (e.g., Alakazam + Dudunsparce). In the *simulator's* meta, a "Crustle" deck reportedly immune to ex/Mega-ex attacks ballooned to ~50% of the field, elevating non-ex attackers and counter decks — a reminder the adjusted-rules sim meta diverges from paper Standard.

Details

State of the art you should draw on (with citations)

Search + learning backbone. AlphaZero (Silver et al. 2018) and MuZero (Schrittwieser et al. 2020) are the base; for this low-simulation-budget, large-action regime the key tool is **Gumbel AlphaZero/MuZero** (Danihelka et al., ICLR 2022, "Policy improvement by planning with Gumbel"), which guarantees policy improvement even with very few simulations by sampling actions without replacement (Gumbel-Top-k) and allocating them via Sequential Halving — ideal given the 600 s clock. Stochasticity is handled by **Stochastic MuZero** (Antonoglou et al., ICLR 2022); large/combinatorial actions by **Sampled MuZero** (Hubert et al., ICML 2021) and **Multiagent Gumbel MuZero** (AAAI 2024). (ICLR) (arxiv)

Imperfect information. Information Set MCTS (Cowling, Powley & Whitehouse, IEEE TCIAIG 2012) searches trees of information sets and mitigates the two documented pathologies of Perfect-Information Monte Carlo (PIMC): **strategy fusion** (a deterministic search makes different choices from the same info set depending on the determinization, using information it should not have) and **non-locality** (payoffs are not recursively defined over subgames; some determinizations are far likelier than others). **Re-determinizing ISMCTS** (Goodman, IEEE CIG 2019, Hanabi) and **MT-ISMCTS** (Cowling et al.) re-sample hidden states per iteration to avoid information leakage and to model opponents' uncertainty (enabling bluffing). Cowling et al. (2012) explicitly applied determinized MCTS to Magic: The Gathering. (White Rose Research Online)

Game-theoretic robustness (directly rewarded by rubric criterion d). CFR family: CFR, CFR+, MCCFR, **Deep CFR** (Brown et al., ICML 2019), Single Deep CFR, DREAM, ESCHER. **Depth-limited solving** (Brown, Sandholm & Amos, NeurIPS 2018) built a master heads-up no-limit hold'em bot on a 4-core CPU by letting the opponent choose among several continuation strategies at the depth limit. **ReBeL** (Brown et al., NeurIPS 2020) unifies RL + search over **public belief states** and provably converges to Nash in two-player zero-sum games. **Player of Games** (Schmid et al. 2021) generalizes across perfect/imperfect information. **DeepNash / R-NaD** (Perolat et al., Science 2022) reaches expert-level Stratego **without any search** via Regularized Nash Dynamics — the strongest evidence that a fast, search-free, equilibrium-seeking net can master a huge imperfect-information game, and a compelling fallback philosophy here.

Card-game-specific. **DouZero** (Zha et al., ICML 2021) shows **Deep Monte Carlo (DMC)** + action encoding (card-matrix action features) + parallel CPU actors beats prior MCTS/CFR DouDizhu bots with no search at inference, surpassing DeltaDou within ~10 days on a single 4-GPU server — the single most relevant template given latency constraints; **DouZero+** and **OADMCDou** (IJCAI 2024) add opponent modeling and oracle guiding (training with perfect

info, annealing to imperfect). The Hearthstone AI Competition (2018–2020) / SabberStone and Legends of Code and Magic / Strategy Card Game AI Competition establish CCG agent + deckbuilding baselines (winning entries used MCTS, Rolling Horizon Evolution, pruned BFS, all paired with a state-evaluation function). Suphx (Mahjong) rounds out the field. (Liner)

Deck construction as a meta-game. **Double Oracle** (McMahan et al. 2003) and **PSRO** (Lanctot et al., NeurIPS 2017) with variants (Pipeline PSRO, XDO, Self-Play PSRO, Efficient PSRO) find an approximate Nash over a strategy/deck population by iteratively adding best responses to the current meta-Nash. For deck diversity, **MAP-Elites with an online deep surrogate** (EvoStone2, ICAROS/USC) set the state of the art in automated Hearthstone deckbuilding; CreativeStone uses a genetic algorithm for deck novelty.

Representation / generalization. Card-as-text encoders using frozen sentence transformers generalize to unseen/teched cards ("Learning With Generalised Card Representations for Magic: The Gathering," arXiv 2407.05879, which appends a sentence-transformer embedding of full card text to numerical/categorical features; DraftEncoder, arXiv 2607.04782, which contextualizes card embeddings with a transformer). Set Transformer (Lee et al., ICML 2019, ISAB + PMA) is the natural encoder over unordered zones. Attention-over-legal-actions / pointer-style heads handle variable action sets without a fixed vocabulary.

Sample efficiency on one GPU. **EfficientZero** (Ye et al., NeurIPS 2021) and **EfficientZero V2** (Wang et al., ICML 2024) add self-supervised (SPR-style) latent consistency, value-prefix prediction, and off-policy correction to reach superhuman Atari-100k in ~2 GPU-hours (EZ-V2 uses a sampling-based Gumbel search); **BBF** (Schwarzer et al. 2023) shows value-based scaling with strong regularization and a high replay ratio. These justify replay-ratio scaling, prioritized replay, and self-predictive auxiliary losses on a single 4090.

Robustness training. AlphaStar-style league training (main agents + main/league exploiters), Fictitious Self-Play, NFSP, and **Prioritized Fictitious Self-Play (PFSP)** directly target the "no over-reliance on specific matchups" criterion by up-weighting the opponents you currently lose to.

Kaggle simulation-competition winning patterns. Lux AI winners combined **imitation learning from top-player replays** with self-play RL (IMPALA / V-trace + UPGO + TD- λ , plus a **frozen-teacher KL term to prevent strategic cycling**), autoregressive action heads, and heavy inference-speed engineering; early reward shaping accelerated learning under sparse win/loss signal. The recurring pattern across Halite/Lux/Kore: IL-from-replays to bootstrap, self-play to refine, a small fast net for submission, and relentless real-ladder A/B testing. This competition uniquely **publishes daily episode replay datasets** — a ready-made imitation corpus.

Recommended architecture: "BeliefSet-DMC"

Design philosophy. The empirical ladder evidence (search often fails to help; simple decks win; model-load and per-move latency matter) plus the theory (DouZero and DeepNash beat huge imperfect-information games without heavy inference-time search) argue against a search-first AlphaZero clone. The winning bet is a **fast, belief-aware, self-play-trained policy** that (1) generalizes across the ~2,000-card pool via card text, (2) explicitly models the hidden opponent state, (3) is robust across matchups via league/PFSP, and (4) uses the engine's

forward model for *targeted* bounded search only where it demonstrably helps, then distills it away for submission.

Input / state encoding.

- *Card embedding (derived, so unseen/teched cards get vectors for free)*: for each card ID, concatenate structured CSV features (one-hot Category/Stage/Type/Weakness/Resistance, numeric HP/Retreat/Damage, per-move energy-cost vectors) with a frozen MiniLM/sentence-transformer embedding of the Effect Explanation + move text, projected 384→64. ≈128-d per card.
- *Zone encoders*: a Set Transformer (ISAB + PMA pooling) over each unordered zone (hand, bench, active, discard, known prizes, stadium), with learned zone-type and status-condition embeddings; opponent zones encoded with count-only masks where hidden.
- *History encoder*: a small GRU/transformer over the last K (`logs`) events (captures tempo, energy attachments used, supporter/stadium usage flags).
- Concatenate → shared trunk transformer/MLP, ~256-512-d.

Heads.

- *Policy (variable action set)*: cross-attention between the trunk state and a learned embedding of each current (`Option`) (OptionType + referenced card embedding + target area/index) → one logit per legal option; softmax over exactly the current (`select.option`), masked to (`minCount...maxCount`). This is attention/pointer over legal actions and needs no fixed action vocabulary.
- *Value*: scalar win-probability, temperature-calibrated post-training.
- *Auxiliary*: (a) **opponent-belief head** — distribution over opponent hand/deck card categories and archetype; (b) **prize-race decomposition** — predict the prize-differential trajectory; (c) **SPR self-predictive latent consistency** for sample efficiency.
- *Size*: target ~1-3M parameters to keep model-load $\approx 2-3$ s (vs. the observed ~8 s for a 1.7M net loaded naïvely — pre-warm and lazy-load) and per-move inference in low milliseconds; export via TorchScript/ONNX + fp16.

Search (used sparingly, then distilled). Belief-conditioned **re-determinized ISMCTS with a Gumbel root** (Danihelka et al. 2022) built on (`search_begin`)/(`search_step`): each simulation draws a fresh determinization of hidden zones from the belief head (avoids strategy fusion); Gumbel-Top-k + Sequential Halving at the root with tiny budgets (8-32 sims); PUCT in the interior; value-net leaf evaluation at a depth limit. Chance nodes use (`manual_coin=True`) to branch coin flips; shuffles/draws are handled by determinized deck order. A **gating rule** invokes search only when the policy's top-1/top-2 margin is small or value entropy is high (the observed failure mode is wasting search on already-clear decisions and on states the value head can't discriminate). Search visit distributions are **distilled back into the policy** (policy iteration via search distillation), so the submitted net is search-free or uses a bounded, well-behaved budget with hard per-move timeout guards.

Training curriculum.

1. **Behavior cloning** from the daily Kaggle episode replays (imitate top- μ agents; value target from game outcomes) — an instant competent baseline, à la Lux winners.
2. **Fixed heuristic/opponent league** of scripted meta pilots (Dragapult/Dusknoir, Mega Lucario, Gholdengo, single-prize control, and the sim-meta "Crustle" counter) plus the public sample agents.
3. **Deep Monte Carlo self-play** (DouZero-style): vectorized multiprocessing CPU actors driving the cabt engine, a single-GPU learner, prioritized replay, replay-ratio scaling, a **frozen-teacher KL term** to prevent strategic cycling (Lux lesson), and reward shaping (prize leads, KOs) for early signal.
4. **League/PFSP**: main agent + exploiters, prioritized toward under-performing matchups, to flatten the matchup matrix (rubric d). Optionally add R-NaD regularization for equilibrium-seeking stability.
5. **Distillation + calibration + export.**

Deck construction (20%). Co-optimize deck and policy. Use **MAP-Elites** over deck space with behavior descriptors (aggro $\leftrightarrow$ control, energy curve, single-prize ratio, tech density) and the value net as a fast surrogate fitness (EvoStone2 pattern), producing a diverse elite archive; then run a **double-oracle/PSRO** step over the elites to select the single deck (or the two alternating submissions) with the best *worst-case* matchup spread. Bias toward simple, low-variance, high-consistency decks — the ladder evidence shows an NN pilots them cleanly. Articulate the chosen deck's concept, engine cards, and key-card roles explicitly for the rubric.

Primary vs fallback.

- *Primary*: BeliefSet-DMC as above (belief + set-transformer + DMC self-play + gated distilled search).
- *Fallback (guarantees leaderboard presence and a defensible report)*: an imitation-cloned policy on a single robust, consistent deck (Dragapult/Dusknoir or a Bellibolt-style Lightning engine), no search, with a hand-crafted legal-move scorer as a safety net. Community evidence suggests this alone plausibly reaches the $\sim 800+$ μ band, and the same belief/robustness analysis still fills a strong Strategy report.

Hypotheses to test (maps to rubric "what hypotheses were tested")

- **H1 (representation)**: card-text embeddings beat one-hot ID embeddings on sample efficiency and on held-out/teched cards.
- **H2 (belief)**: belief-conditioned determinization beats uniform determinization (measured by belief calibration + win rate).
- **H3 (search gating)**: gated bounded search > always-search > never-search on ladder μ and local Elo.
- **H4 (deck simplicity)**: a simple consistent deck piloted by the NN beats a complex combo deck (tests the community claim directly).
- **H5 (robustness)**: league/PFSP flattens the matchup matrix vs pure self-play (variance of per-matchup win rate).

- **H6 (distillation/latency):** the distilled fast policy retains $\geq 90\%$ of the search agent's strength while meeting load/latency limits.
- **H7 (deck robustness):** the double-oracle-selected deck has a flatter matchup spread than the single best-average deck.

Evaluation protocol (demonstrates consistency + no matchup over-reliance)

A local TrueSkill/Elo ladder among agent versions and a fixed panel of meta bots; **seed-controlled repeated matches** (fix shuffle/coin seeds) to isolate policy variance from luck; matchup matrices with Wilson/bootstrap 95% CIs; decision-entropy-over-turn curves; belief-model reliability diagrams; an **exploitability proxy** via a best-response agent trained against the frozen policy (PSRO-style). Crucially, corroborate every local finding with **real-ladder A/B** (the community's hard-won lesson that local sims mispredict ladder rank) — spend your 5 daily submissions deliberately.

Report plan (10%) — section-by-section ($\leq 2,000$ words + media gallery)

1. Problem & framing (imperfect-info POMDP; cabt specifics).
 2. Approach & rationale (why DMC + belief + gated search, why not search-first).
 3. State/action representation.
 4. Belief modeling.
 5. Training curriculum & league.
 6. Deck concept & co-optimization.
 7. Experiments & hypothesis results.
 8. Robustness & consistency evidence.
 9. Limitations & future work.
- Figures/tables to produce:** matchup win-rate heatmap with CIs; Elo/ μ -over-training curve; ablation bar charts (H1-H7); belief-model calibration/reliability plot; decision-entropy vs turn; game-length distribution; prize-race trajectory plots; card-usage/importance attribution (attention weights); saliency/attention visualization on a key turn; search-gating trigger histogram; deck MAP-Elites archive heatmap. Because words are capped at 2,000, **push detail into well-captioned figures** — the rubric explicitly rewards visuals.

Risk register

Risk	Impact	Mitigation	Go/No-Go
cabt API / adjusted-rules surprises	High	Build engine harness + legality tests week 1; pin kaggle-environments==1.30.1	Working self-play loop by Jul 27 or fall back to imitation-only
Action-space / decision-count explosion	Med	Attention-over-options head (no fixed vocab); option masking	If BC won't converge by Aug 2, simplify heads
Self-play collapse / cycling	Med	Frozen-teacher KL, league/PFSP, optional R- NaD reg	If μ stalls by Aug 9, freeze to best checkpoint
Latency / model- load timeout	High	≤ 3 M params, ONNX/fp16, lazy+pre-warm load, per-move budget guard, always return a legal fallback	Validation self-game must pass on every submit
Local $\neq$ ladder	High	Daily real-ladder A/B; keep 2 alternating submissions	—
Deck too complex to pilot	Med	Bias MAP-Elites to low-variance decks; test H4	Prefer simple deck if it out- μ s the combo
Deadline slip (agent locks ~Aug 16)	High	MVP on ladder by Jul 27; freeze agent Aug 14; report-only afterward	Hard freeze at the Aug 16 UTC deadline

Tooling & repo

PyTorch; sentence-transformers (MiniLM) for card text; a custom lightweight self-play loop with `multiprocessing`/`ray` CPU actors around the cabt engine (RLlib is overkill and awkward with a bespoke ctypes engine); `open_spiel` for reference CFR/PSRO baselines and exploitability tooling; PettingZoo-style wrapper only if convenient; Weights & Biases for tracking; the `kaggle` CLI for daily episode pulls and submissions; TorchScript/ONNX for export. Suggested repo: `core/` (card featurization, obs model, engine wrapper), `agents/`, `train/` (BC, DMC, league), `deck/` (MAP-Elites, double-oracle), `eval/` (ladder, matchup matrices, exploitability), `episodes/` (replay pull/parse), `dist/` (packaged submissions), `report/` (figures).

Week-by-week timeline (Jul 23 - Sep 13, 2026)

Week / dates	Theme	Concrete deliverables	Ladder action	Gate
W0: Jul 23-27	Engine + MVP	cabt harness (v1.30.1), CSV→feature pipeline, legal- move heuristic scorer on a simple deck, legality/crash tests	Submit MVP-1	Non-crashing agent on ladder by Jul 27; else ship imitation- only
W1: Jul 28- Aug 4	Imitation + representation	Daily episode puller; BC of set-transformer/text- embedding policy+value+belief heads; run H1	Submit MVP-2 (BC net)	BC $\mu \geq$ heuristic μ ; else keep heuristic live while debugging
W2: Aug 5- 11	Self-play + league	DMC self-play (CPU actors + GPU learner), frozen-teacher KL, PFSP league, belief- conditioned training; run H2 , H5 ; daily A/B	Submit league agent	League beats BC $\geq 55\%$ H2H and improves worst- case matchup
W3: Aug 12-16	Search, distill, deck, FREEZE	Gated re-determinized Gumbel-ISMCTS via forward model (H3, H6); deck via MAP-Elites + double-oracle (H4, H7); distill, calibrate, export	Lock 2 best alternating submissions before Aug 16 UTC deadline	Distilled net within 10% of search agent and passes latency; else submit search-free net
W4-5: Aug 17- Sep 6	Analysis & ablations	Full eval protocol: matchup matrices w/ CIs, exploitability proxy, calibration, entropy/attribution; produce all figures	(ladder converging/locked)	All 11 figures drafted by Sep 6
W6+: Sep 7- 13	Write & polish	$\leq$ 2,000-word report to the 70/20/10 rubric, assemble media gallery, tighten rationale narrative	—	Submit Strategy report before Sep 13 UTC

Recommendations (staged, with thresholds)

- Stage 0 (Jul 23-27) — get on the board.** Stand up the engine harness, card featurization, and a legal-move-scoring heuristic on a simple consistent deck; submit MVP-1.
Threshold: non-crashing ladder entry by Jul 27; if the engine fights you, ship imitation-only and defer self-play.

- **Stage 1 (Jul 28-Aug 4) — imitation + representation.** Behavior-clone from daily episodes; run H1; submit the BC net. *Threshold:* BC $\geq$ heuristic μ .
- **Stage 2 (Aug 5-11) — self-play + league.** DMC self-play with frozen-teacher KL and PFSP; run H2, H5; daily A/B. *Threshold:* league beats BC $\geq$ 55% and flattens worst-case matchups.
- **Stage 3 (Aug 12-16) — search, distill, deck, FREEZE.** Add gated Gumbel-ISMCTS (H3, H6), finalize deck (H4, H7), distill/calibrate/export, and lock the two alternating submissions before the Aug 16 UTC lock. *Threshold:* distilled net within 10% and latency-safe.
- **Stage 4 (Aug 17-Sep 6) — analysis & ablations.** With the agent frozen, run the full evaluation protocol and build every figure.
- **Stage 5 (Sep 7-13) — write & polish.** Draft the report emphasizing rationale over raw win rate; assemble the gallery; submit before the Sep 13 deadline.
- **What would change the plan:** if by Aug 9 self-play hasn't beaten BC, cut search and league and ship the best imitation net on a simple deck (still a strong, analyzable Strategy entry). If belief modeling degrades win rate (H2 fails), drop it from the *agent* but keep it as an analysis instrument in the *report*. If model-load ever risks timeout, shrink params and re-export before anything else.

Caveats

- The competition uses "official rules uniquely adjusted for the tournament," and the materials warn "some simulator behavior differs from the official Pokémon TCG rules" — validate all deck/legality assumptions against the actual engine, not paper Standard.
- The ~2,000-card count and exact per-move time limit are reported but not fully confirmable from official pages; verify against `all_card_data()` and the live Kaggle rules page. The 197.7 MiB bundle cap and the 600 s/10 min per-player clock are corroborated by EdTech Innovation Hub, AICU, and Shane the Gamer, but confirm the precise UTC cutoffs and any per-move cap on the live rules page before each freeze.
- Ladder- μ figures (~530-880) and the "most of the field is rule-based" observation come from community repos and one Kaggle discussion analysis, not official statistics — directional only.
- The meta shifts fast, and the simulator's meta (e.g., the "Crustle" phenomenon) can diverge from the physical-game meta; re-scout the ladder weekly.
- All dates are as reported (JST anchors with UTC corroboration); the Simulation deadline is best treated as the **Aug 16 11:59 p.m. UTC** hard lock and the Strategy deadline as **Sep 13 UTC**.